

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	V.JAGADISH	Reg. No.:	192521352
Section / Batch:		Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Industry Problem Solving Task

A city must optimize traffic signal timings to reduce congestion and waiting time at intersections. Analyze how optimization algorithms can be applied to control signal phases. Identify constraints such as traffic density and peak hours. Design a solution using dynamic programming techniques. Discuss real-time implementation feasibility. Evaluate system efficiency. Interpret results in urban traffic management.

Code of Conduct

I Jagadish 192521352 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: jagadish

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%				
Application of Engineering Concepts	25%				
Solution Design	25%				
Use of Tools	15%				
Analysis	10%				
Professionalism	5%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Algorithm: Smart Traffic Signal Control Optimization using Dynamic Programming

Step-by-Step Algorithm

1. **Start**
2. Read the traffic densities of all signal phases.
3. Let n be the number of phases.
4. Divide the total available time by the unit:

$$T = \frac{total_time}{unit}$$

5. Create a DP table:
 - o $dp[i][t]$ = minimum waiting cost for the first i phases using t time units.
6. Initialize:
 - o $dp[0][0] = 0$
 - o All other values are infinity.
7. For every traffic phase i from 1 to n :
 - o For every available time t from 0 to T :
 - Try every possible allocation a from 0 to t .
8. Calculate the allocated time:

$$time = a \times unit$$

9. Calculate the waiting cost:

$$cost = density[i - 1] \times \frac{100}{time}$$

If $time = 0$, use the density as the cost.

10. Calculate total cost:

$$total = dp[i - 1][t - a] + cost$$

11. If $total < dp[i][t]$:
 - Update $dp[i][t] = total$
 - Store a in $choice[i][t]$.
12. After filling the DP table, start from $dp[n][T]$.
13. Backtrack using the choice table to find the optimal time for every phase.
14. Multiply each allocated unit by unit to obtain the signal duration in seconds.
15. Display:
 - Optimal duration for each phase
 - Minimum waiting cost
16. **Stop**

For the picture's example

Given:

Densities = [30, 45, 20, 50]

Unit = 5 seconds

The algorithm distributes the available signal time among the **4 phases** so that the overall waiting-cost function is minimized.

Main DP idea:

Try every possible time allocation for every traffic phase, store the minimum cost for each state, and finally backtrack to obtain the optimal signal timings.

Time Complexity:

$$O(nT^2)$$

where n is the number of traffic phases and $T = total_time / unit$.

Space Complexity:

$$O(nT)$$



QUESTIONS

Q1

Smart Traffic Signal Control
Optimization (Dynamic
Programming)
100 marks

0/1 answered

Q1 Smart Traffic Signal Control Optimization (Dynamic Programming)

100 marks

A city must optimize traffic signal timings to reduce congestion and waiting time at intersections. Analyze how optimization algorithms can be applied to control signal phases. Identify constraints such as traffic density and peak hours. Design a solution using dynamic programming techniques. Discuss real-time implementation feasibility. Evaluate system efficiency. Interpret results in urban traffic management.

Your Code

```
1 def min_waiting_cost(densities, total_time, unit=5):
2     n = len(densities)
3     T = total_time // unit
4     dp = [[float('inf')] * (T + 1) for _ in range(n + 1)]
5     choice = [[0] * (T + 1) for _ in range(n + 1)]
6     dp[0][0] = 0
7
8     for i in range(1, n + 1):
9         for t in range(T + 1):
10            for a in range(t + 1):
11                if dp[i-1][t-a] == float('inf'):
12                    continue
13                time = a * unit
14                cost = densities[i-1] * (100 / time) if time else densi
15                total = dp[i-1][t-a] + cost
16                if total < dp[i][t]:
17                    dp[i][t] = total
18                choice[i][t] = a
```

Input

stdin input

Output

Densities: [30, 45, 20, 50]
Phase 1: 30s (density=30)
Phase 2: 35s (density=45)
Phase 3: 20s (density=20)
Phase 4: 35s (density=50)
Min Waiting Cost: 471.43